**Std::Unique_Lock Move-Assignment Operator**

Transfers ownership of a lock from one `std::unique_lock` object to another `std::unique_lock` object.

*Declaration*

```
unique_lock& operator=(unique_lock&& other) noexcept;
```

*Effects*

If `this->owns_lock()` returns `true` prior to the call, calls `this->unlock()`. If `other` owned a lock on a mutex prior to the assignment, that lock is now owned by `*this`.

*Postconditions*

`this->mutex()` is equal to the value of `other.mutex()` prior to the assignment, and `this->owns_lock()` is equal to the value of `other.owns_lock()` prior to the assignment. `other.mutex()==NULL, other.owns_lock()==false`.

*Throws*

Nothing.

**Note**

`std::unique_lock` objects are *not* `CopyAssignable`, so there's no copy-assignment operator, only this move-assignment operator.

**Std::Unique_Lock Destructor**